

Streaming Live RAG - Architecture Brief

Team: UNDEAD **College:** SRM Institute of Science and Technology

Members: KA Mohammed Ziyen (RA2411026010528), Subham Sankhala (RA2411031010082)

Theme: 4 - Streaming Live RAG

1. Problem and Design Goal

The supplied Theme 4 guide defines Streaming Live RAG as real-time incremental retrieval, multi-intent decomposition, and state-preserving answer refinement. The design addresses four behaviors: start retrieval before an utterance is complete when intent is stable, split compound requests into independent retrieval tasks, refine an existing answer when a late constraint arrives, and suppress retrieval for presentation-only requests.

2. System Pipeline

Stage	Responsibilities
1. Retrieval Controller	Classifies each transcript chunk as WAIT, RETRIEVE, or NO_RETRIEVAL; identifies provisional and late-refinement triggers.
2. Multi-Intent Decomposer	Extracts independent search-ready subqueries and routes them in parallel.
3. Corpus Retrieval & Fusion	Runs dense + sparse retrieval, reranking and deduplication; fuses evidence across subqueries.
4. Session-Aware Synthesis	Updates answer state, preserves valid prior facts, checks grounding, and adds explicit uncertainty.
5. Observability	Writes timestamped JSONL events for decisions, queries, latency, source mappings, and answer versions.

3. Incremental Retrieval Controller

The controller uses the current chunk, active session answer state, and lightweight stability/search-intent signals. Very short fragments return WAIT. Search nouns or question forms return RETRIEVE once there is enough semantic stability. Formatting-only requests such as “repeat the last answer in bullet points” return NO_RETRIEVAL. When a session already has an answer and a new chunk expresses a constraint (for example “actually”, “international”, “after travel”), the controller emits a late_refinement retrieval event.

4. Multi-Intent Decomposition and Parallelism

Compound requests are decomposed conservatively. The sample flow “capacity, cancellation policy and catering options” becomes three search-ready queries. The implementation uses a thread pool to issue subqueries concurrently. A single full-query fallback is retained when the decomposer cannot confidently identify multiple distinct intents, avoiding over-fragmentation.

5. Retrieval and Evidence Fusion

The local implementation uses a sparse TF-IDF retriever and an offline dense latent projection based on Truncated SVD. Scores are normalized and blended as **hybrid = alpha * dense + (1 - alpha) * sparse**. Lightweight lexical overlap provides a reranking signal. Evidence appearing across multiple subqueries receives a small coverage bonus. Near-duplicate chunks are removed before synthesis.

6. Session-Aware Refinement

Each session keeps only active conversation state: transcript chunks, the current answer, source evidence, citations, constraints, and answer version. A late detail does not clear state. The engine retrieves only the delta query, merges new evidence into the existing evidence set, and generates the next answer version. Unaffected evidence remains available so the response does not “restart”.

7. Grounding and Uncertainty

The application creates citation strings directly from corpus metadata in the form [Doc_ID §Section]. The model, when configured, receives only retrieved evidence. The application appends verified source citations and can return an explicit uncertainty message when no supporting corpus evidence is available. No web search is used by the retrieval engine.

8. Streaming and API Surface

Endpoint	Purpose
GET /health	Readiness check returning {"status":"ok"}.
POST /v1/query	Process a complete query and return the current answer state.
POST /v1/stream	Process timestamped transcript chunks and stream state/token events over SSE.
POST /v1/corpus/append	Demo event-driven ingestion endpoint for adding new corpus records.

9. Observability

Telemetry records include event timestamp, session ID, transcript timestamp, controller decision, reason, trigger, subqueries, retrieval latency, source citations, answer version, and citation support ratio. JSONL provides a simple replayable audit stream for benchmarking and debugging.

10. Reproducibility and Deployment

The repository contains pinned requirements, Dockerfile, docker-compose.yml, automated tests, a local sample corpus, benchmark scripts, and a Streamlit interface. A clean environment can be started with docker compose up --build. The architecture is intentionally self-contained so the official corpus can later replace the demonstration corpus through CORPUS_PATH.

11. Trade-offs

Decision	Reason / Trade-off
Hybrid local retrieval	Keeps the demo reproducible without an external vector database; production can replace it with a persistent vector store.
Heuristic controller	Low latency and explainable; a production version can combine rules with a small intent classifier.
Optional LLM	The default deterministic synthesizer makes the repository runnable without credentials; a Gemini adapter is available v
In-memory session store	Simple for a single-process hackathon demo; a production multi-instance system would use a shared session store.

12. Risks and Mitigations

Premature retrieval: controller WAIT state and minimum stability rules. **Over-fragmentation:** single-query fallback. **Contradictory/duplicate evidence:** fusion and deduplication. **Unsupported claims:** corpus-only synthesis and citations. **Unnecessary search:** NO_RETRIEVAL suppression. **Slow repeated processing:** targeted late-refinement retrieval rather than full restart.

13. Alignment with the Theme 4 Evaluation Gates

Gate	Implementation mapping
G1 Reproducibility	Dockerfile, compose file, pinned dependencies, tests, one-command startup.
G2 Early Retrieval	Controller supports provisional RETRIEVE before final transcript completion.
G3 Multi-Intent	Decomposer extracts multiple intent classes and performs parallel retrieval.
G4 Factual Grounding	Citations are derived from retrieved source metadata and uncertainty is explicit.
G5 Session Refinement	Late constraints trigger delta retrieval and increment the answer version without clearing state.
G6 Telemetry	Structured JSONL event logs cover the end-to-end decision/retrieval/synthesis path.

The thresholds themselves should be validated on the official Samsung corpus/test set before being claimed as passed. The included local benchmark is an engineering smoke test, not an official score.